

The banner features the Adobe logo in red on the left. The main text 'Adobe University Hackathon 2026' is in white. The background is a dark blue gradient with vibrant, colorful light streaks in shades of pink, purple, and blue. On the right side, there is a grid of small, glowing squares in various colors, resembling a digital or data visualization theme.

Adobe

Adobe University Hackathon 2026

Round 4 — Prototype Showcase (Live Agent Demo)

1. What Round 4 is

Round 3 asked you to **build** an Agent Skill Marketplace: a headless engine that audits any website for AI-discoverability and on-site-engagement problems and emits a report of findings + suggested actions.

Round 4 asks you to **show it works, live, and prove it is reproducible**.

You do this with a single recorded video plus your (unchanged) Round 3 submission and a small reproduction manifest so we can re-run your demo ourselves and corroborate every finding.

You do not need to build a graphical user interface. A CLI-based session in which an AI agent runs your marketplace and reports its findings is exactly what we want to see. The "interface" is the agent interaction itself — not a web app.

2. What you submit

- **A recorded video** (≤ 5 minutes, hard cap) — see Section 3.
- **A reproduction manifest** — a single plain-text file named "REPLAY_YourTeamName.txt", submitted alongside the video — see Section 5.

We already have your Round 3 marketplace and will reuse that exact package — you do not resubmit it. The live run in your video must exercise that same submitted marketplace; if the engine shown differs from what you submitted in Round 3, the submission fails on integrity grounds (Section 6).

The video is the pitch; REPLAY_YourTeamName.txt is the proof. Judges will replay your demo on their own machine, including on a website they choose, not only the URLs you used.

3. The video (≤ 5 minutes, hard cap)

Two parts, with a fixed per-part budget so neither half starves:

- **Part 1 — methodology:** ≤ 3 minutes
- **Part 2 — trial run:** ≤ 2 minutes

You may crop out the uninteresting parts of the trial run (e.g. long waits while the agent fetches or thinks) to fit Part 2 inside 2 minutes — trimming idle time is expected and fine. What you may not do is splice together a run that did not happen: the URL you enter and the findings that appear must be from one real, continuous run (see the integrity gate in Section 6).

Part 1 — Thought process (≤ 3 minutes)

Explain the reasoning behind the skills you built:

- **How you discovered the findings** — the field research behind your signals: what separates websites that AI assistants cite well from ones they ignore or misrepresent, and how you distilled that into concrete, repeatable checks.
- **How you assigned severities** — the logic that decides how much each problem hurts discoverability or engagement.
- **How you derived the suggested actions** — why each fix is mechanism-sound and how you prioritized them.

Anchor every claim to your code. When you describe a check, a severity rule, or a fix template, point to where it lives in the submission (e.g. the script / reference file that implements it). Narration that cannot be traced to the submitted code earns no credit — a claim is only as real as the code behind it.

Part 2 — Trial run (≤ 2 minutes)

Run your marketplace live, on camera, against a site the audit has never seen:

- **Enter the URL on camera** — the run must be real and continuous. You may trim idle wait time, but there must be no fabricated jump from the URL you type to the findings that appear. Interaction, not a pre-rendered static report.
- **Let the agent work in the CLI** — show the agent (via your chosen harness) invoking your entrypoint skill, fetching, reasoning, and producing findings.
- **Drill into the findings** — walk through real evidence for at least one finding (e.g. raw-HTML-vs-rendered-DOM diff, missing/invalid JSON-LD, stale or contradictory dates, facts locked in an image/PDF), its severity, and the suggested fix. Show that fixes can be sorted/prioritized by impact.

- **State your harness and model on camera** — say which LLM and which agent harness are driving the run (details also captured in `REPLAY_YourTeamName.txt`, Section 5).
- **One line on wiring** — briefly confirm the session is really invoking your Round 3 marketplace entrypoint, not a hardcoded demo.

Everything shown must come from the submitted engine, running live, so that a judge re-running the same steps reaches the same substance.

4. Harness and model

Your marketplace is provider-neutral (agentskills.io format), so the same skill behaves differently under different runtimes. To make your live run reproducible, you must disclose exactly what drove it:

- **LLM model** — name and version/date (e.g. "GPT-5", "Claude Opus 4.6", "Gemini 2.5 Pro"). "We used GPT" is not reproducible.
- **Agent harness** — name and version (e.g. Claude Code, Codex, Copilot CLI, Antigravity, OpenCode, or a custom runner).
- **Exact invoke command(s)** — how the harness is pointed at your marketplace entrypoint, from a clean checkout through to the emitted report.

If you use a closed or paid harness that judges may not be able to run identically, document a fallback run path. Judge corroboration weighs substance (are the real problems surfaced with evidence?), not exact string-for-string output, since different models phrase findings differently.

5. Reproduction manifest — `REPLAY_YourTeamName.txt`

A single plain-text file submitted alongside the video, so judges open one known file and can replay your run. Fixed, labeled fields:

```
TEST_SITE_URLS: https://example.com/, https://example.com/pricing
LLM_MODEL:      Claude Opus 4.6 (2026-05)      # name + version/date
AGENT_HARNESS:  Claude Code v1.2.3             # name + version
ENV_SETUP:      pip install -r requirements.txt  # deps / how to install
TEST_STEPS:
1. <exact command that points the harness at your entrypoint skill>
```

2. <the prompt / input you give the agent, verbatim>
3. <where the report is written and how findings appear>
EXPECTED_OUTPUT: path/to/report.json # where the audit lands

Rules:

- **Mandatory and parseable.** Missing, blank, or unparseable fields mean judges cannot corroborate your run → the run is treated as unverified (Section 6).
- **Test site URLs** must be exact, full URLs — not "a SaaS site." Judges will also run a URL of their own choosing to test generalization.
- **Test steps** must be copy-pasteable, in order, reproducible from a clean checkout on the disclosed harness + model.
- **Consistency with the video.** The harness, model, and URLs in REPLAY.txt must match what the video shows. A mismatch is an integrity failure.

6. How Round 4 is judged

Rubric (out of 100)

Criterion	Looking for	Weight
Reasoning (Part 1)	Signal discovery, severity logic, and fix derivation are sound, non-obvious, and traceable to the submitted code	35
Live behaviour (Part 2)	On an unseen URL, the live agent run surfaces real, evidence-backed findings — few misses, few false positives	35
Reproducibility, quality & hygiene	Judges can replay the run via the disclosed harness + model + REPLAY.txt and reach the same substance; findings and actions hold up under a judge-chosen URL (correct, specific, mechanism-sound, prioritized); harness/model fully disclosed, REPLAY.txt well-formed, engine self-consistent with the video, deterministic	30

Gates (applied regardless of rubric score)

- **Integrity floor.** Fabricated findings, a demo engine that differs from the submitted engine, a trial run spliced from separate takes to fake a result (as opposed to merely trimming idle wait time, which is allowed), or a `REPLAY_YourTeamName.txt` that contradicts the video → forced to the bottom. A demo that lies cannot score its way up.
 - **Unverified floor (ran-blocked).** If judges cannot reproduce the run — missing/unparseable `REPLAY_YourTeamName.txt`, no working invoke path, engine won't execute on the disclosed harness — behaviour is unverified and the submission sinks, tagged ran-blocked.
 - **Nondeterminism flag.** If replay produces materially different findings than the video (beyond expected model-phrasing variance), the submission gets a second independent review before scoring.
-

7. Scope & guardrails

- **Recommend-only.** The marketplace still only audits and reports; no skill alters a live site. Everything runs read-only. Respect `robots.txt`.
 - **Same engine as Round 3.** Round 4 is a demonstration + reproducibility round, not a rebuild. The live run must exercise your submitted Round 3 marketplace.
 - **AI assistance allowed** — building and running agent tooling with an agent is the point.
 - **No GUI required.** A CLI agent session satisfies the "interface / interaction" expectation in the public problem statement.
-

Good Luck!